Web Application Security Assessment Report

Target Application: PrestaShop Web Application
Assessment Type: Attack and Defense Security Validation

Author: Nweke Nnaemeka Anthony
Role: Cybersecurity Intern
Date: 13-03-2026

Executive Summary

This report documents a security assessment conducted on a deployed PrestaShop web application to evaluate its resilience against common web-based attacks.The objective of the assessment was to:

- Identify potential vulnerabilities in the web application
- Simulate real-world attack techniques
- Implement defensive security mechanisms
- Validate the effectiveness of the deployed protection systems

The assessment involved testing for common vulnerabilities such as directory exposure, SQL injection, and security misconfigurations. A Web Application Firewall was deployed to defend the application from malicious requests.The results indicated that the deployed protection mechanisms successfully blocked malicious payloads while identifying minor configuration improvements.

Assessment Scope

Target Application:	PrestaShop
Target URL:	http://192.168.206.139/prestashop
Server OS:	Ubuntu
Web Server:	Apache
Attacker System:	Kali Linux

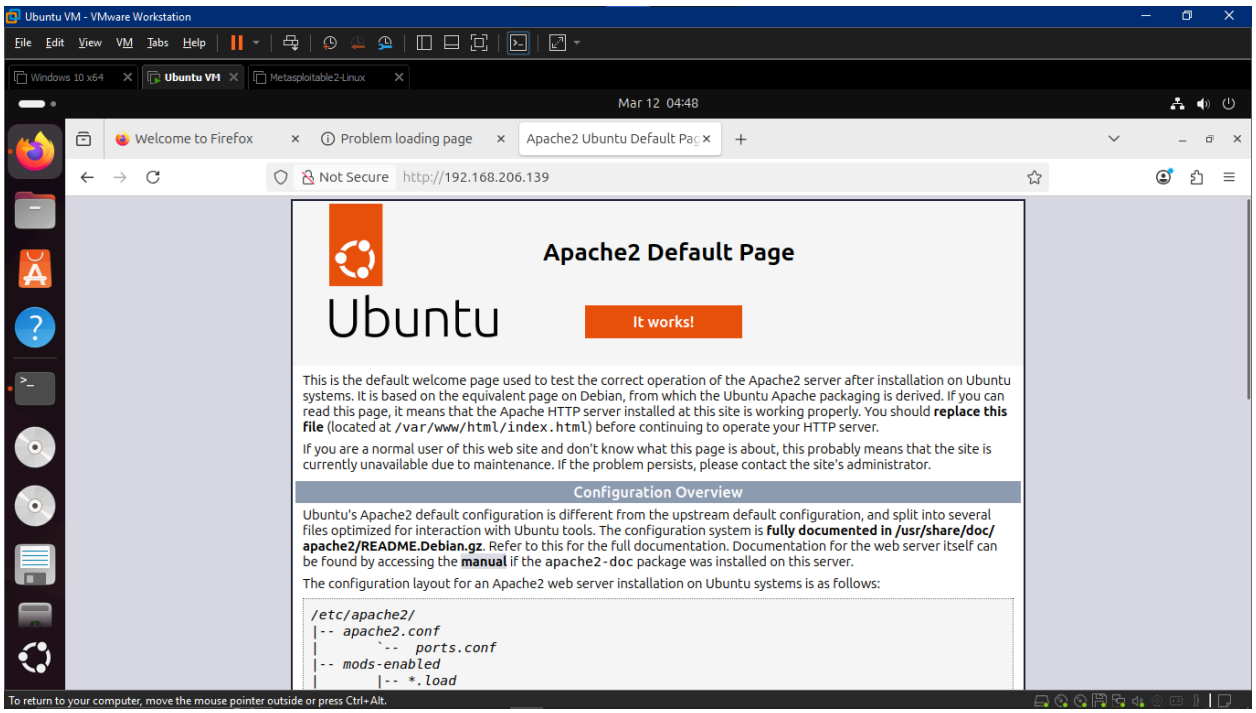
Tools Used in Assessment

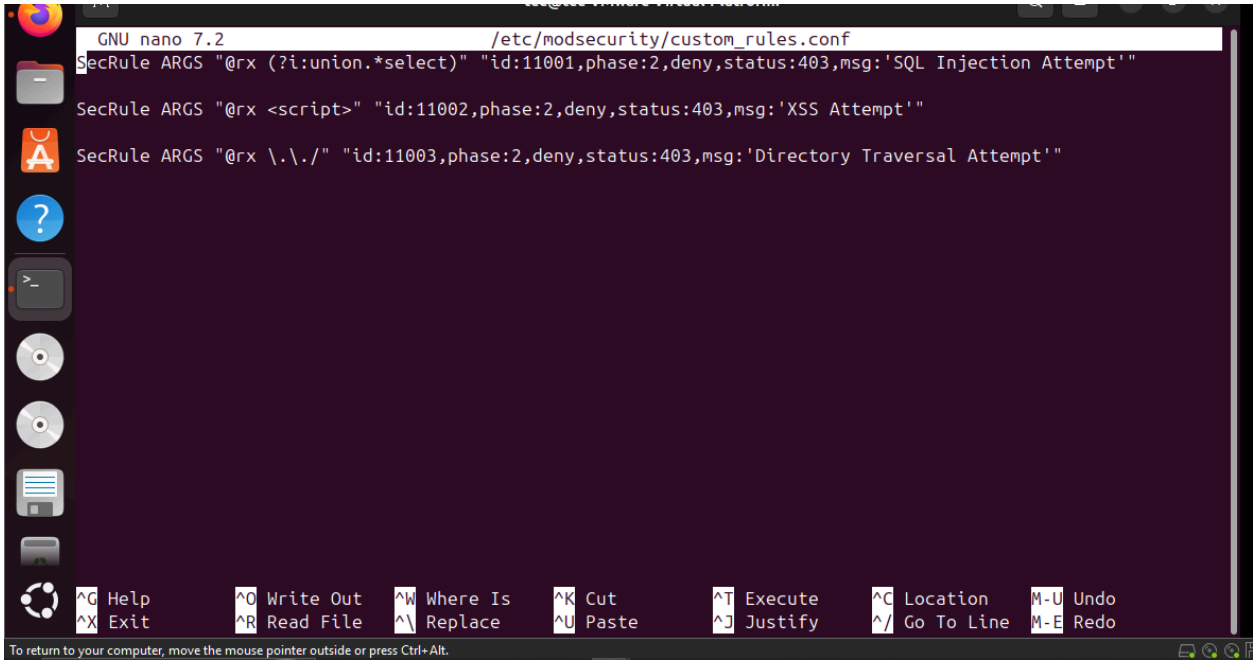
Tools	Purpose
sqlmapSQL	injection testing
Gobuster	Directory enumeration
OWASP ZAP	Automated vulnerability scanning
curl	Manual request testing
ModSecurity	Web Application Firewall

Step 1. Environment Setup

The web application was deployed on an Ubuntu Desktop running Apache HTTP Server. The application used was PrestaShop, an open-source e-commerce platform commonly used for online stores.The attacker environment consisted of Kali Linux running inside a virtual machine. Security testing tools available in Kali Linux were used to perform attack simulations.To defend the application, a Web Application Firewall (ModSecurity) was configured with custom rules to detect and block malicious requests.

Below are the screenshots of Ubuntu Desktop running Apache HTTP Server and a Web Application Firewall (ModSecurity) configured with custom rules to detect and block malicious requests.





Step 2. Attack Simulation and Testing

Attack 1 Directory Enumeration

Directory enumeration was performed using Gobuster to discover hidden directories within the web application.

- The Command used:
- Sudo gobuster dir -u http://192.168.206.139/prestashop -w /usr/share/wordlists/dirb/common.txt

The scan revealed several accessible directories including:

- admin
- modules
- config
- themes

These directories contain important application components and may expose sensitive information if improperly secured. Attached below is the screenshot of Gobuster Scan Result

Attack 2 SQL Injection Testing

SQL injection testing was conducted using sqlmap against the product parameter in the application/web

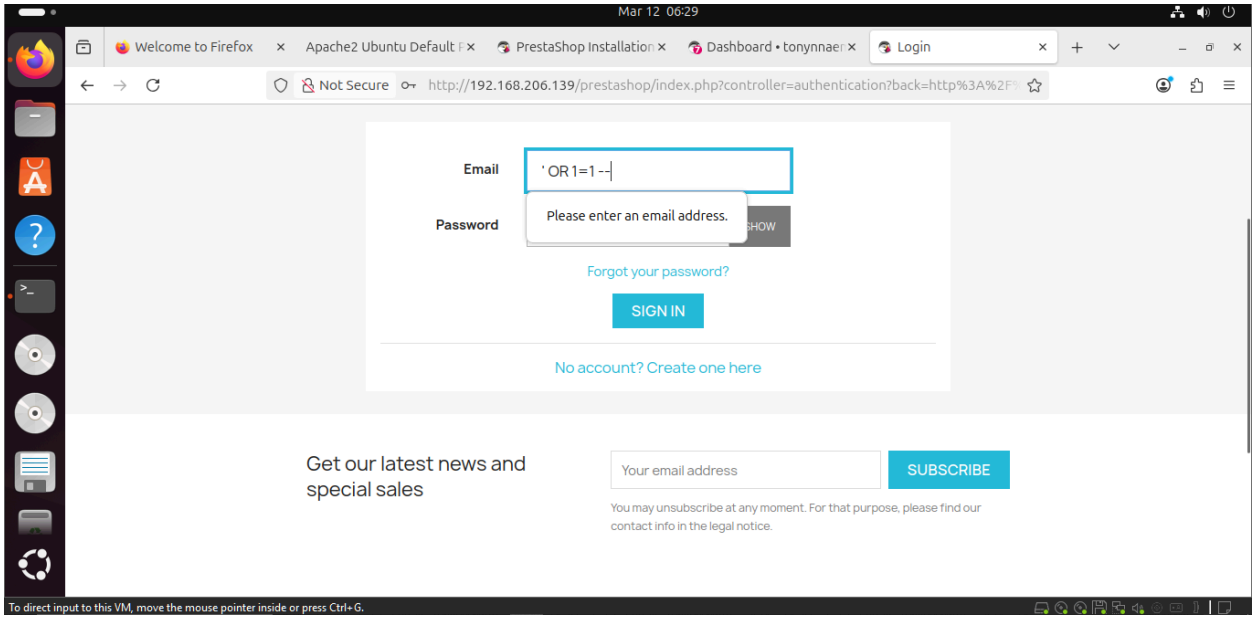
- Command used:
- Sudo sqlmap -u "http://192.168.206.139/prestashop/index.php?id_product=1" --batch
- ‘OR 1=1- -

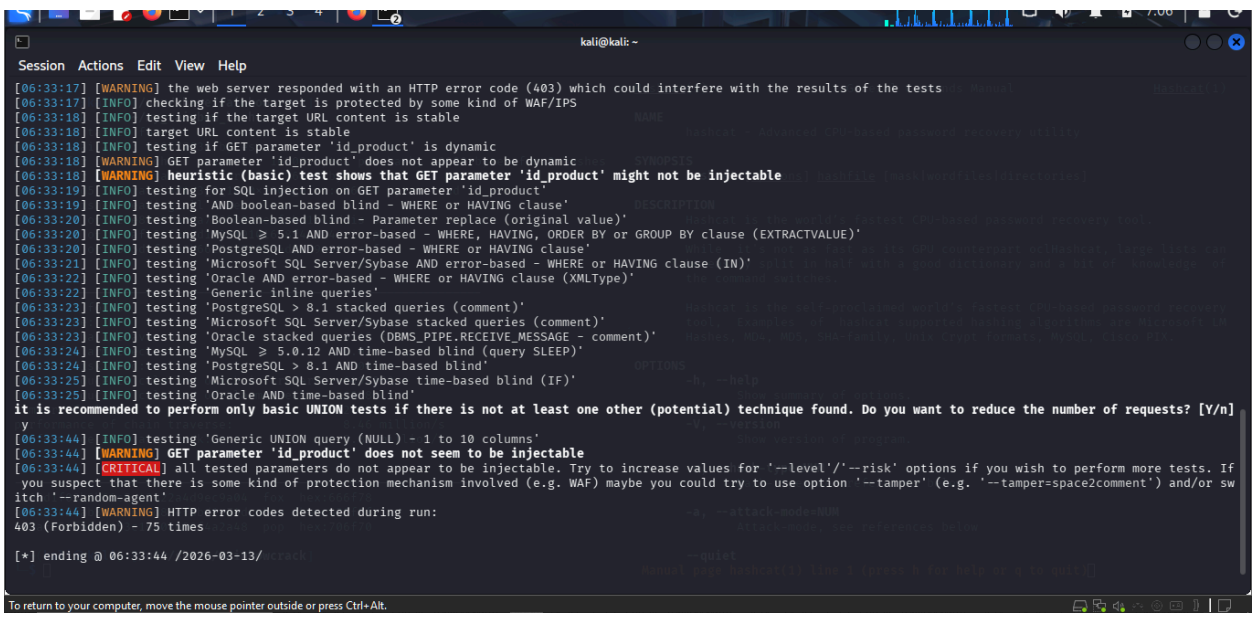
The testing included several injection techniques:

- Boolean-based SQL injection
- Error-based SQL injection
- Time-based SQL injection
- UNION query injection

Results indicated that the tested parameter was not vulnerable to SQL injection.Additionally, multiple HTTP 403 responses were returned during testing, suggesting that malicious requests were blocked by the Web Application Firewall. Attached below is the screenshots of SQLMap

Output





Attack 3 – Automated Vulnerability Scanning

Automated vulnerability scanning was performed using OWASP ZAP.

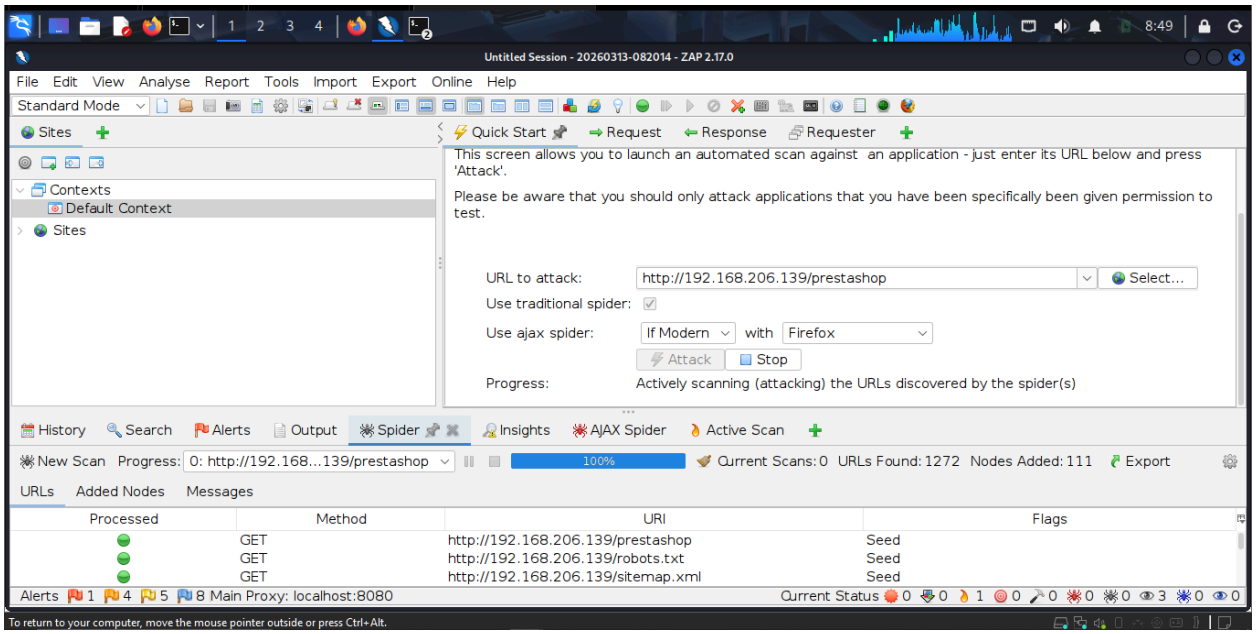
The target scanned is

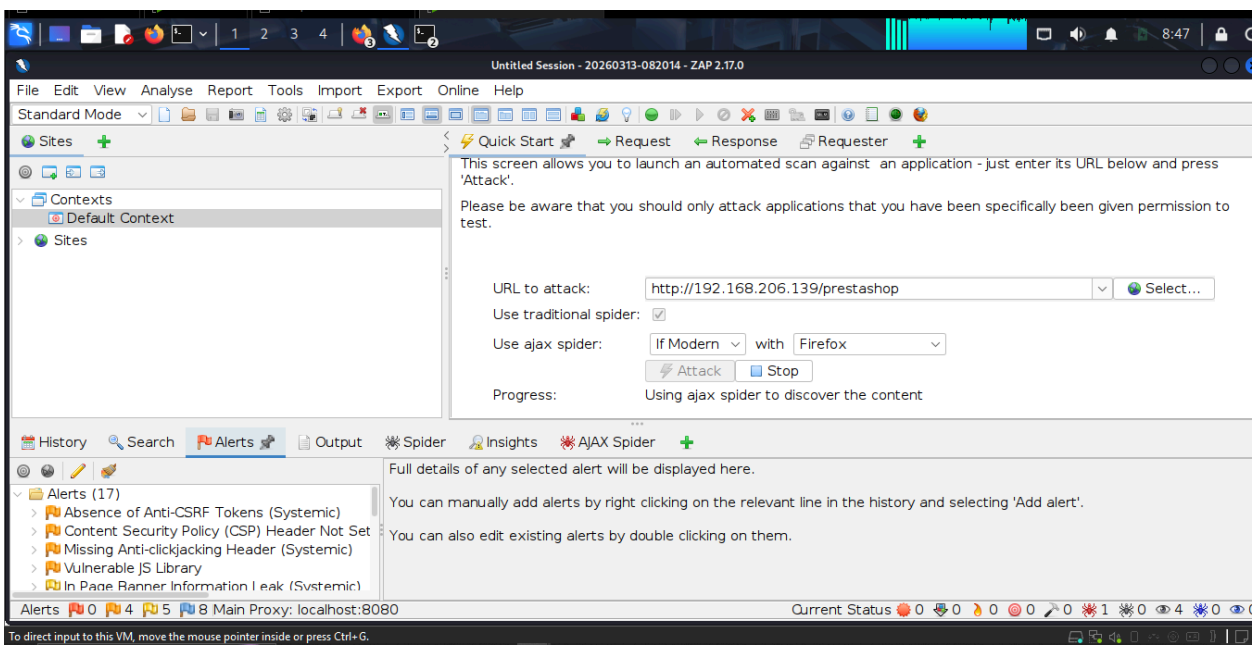
http://192.168.206.139/prestashop

The scanner identified several low to medium risk issues including:

- Missing security headers
- Cookie security configuration issues
- Information disclosure warnings

These findings indicate areas where the application's security posture can be improved, attached below is the screenshot of OWASP ZAP Scan Results





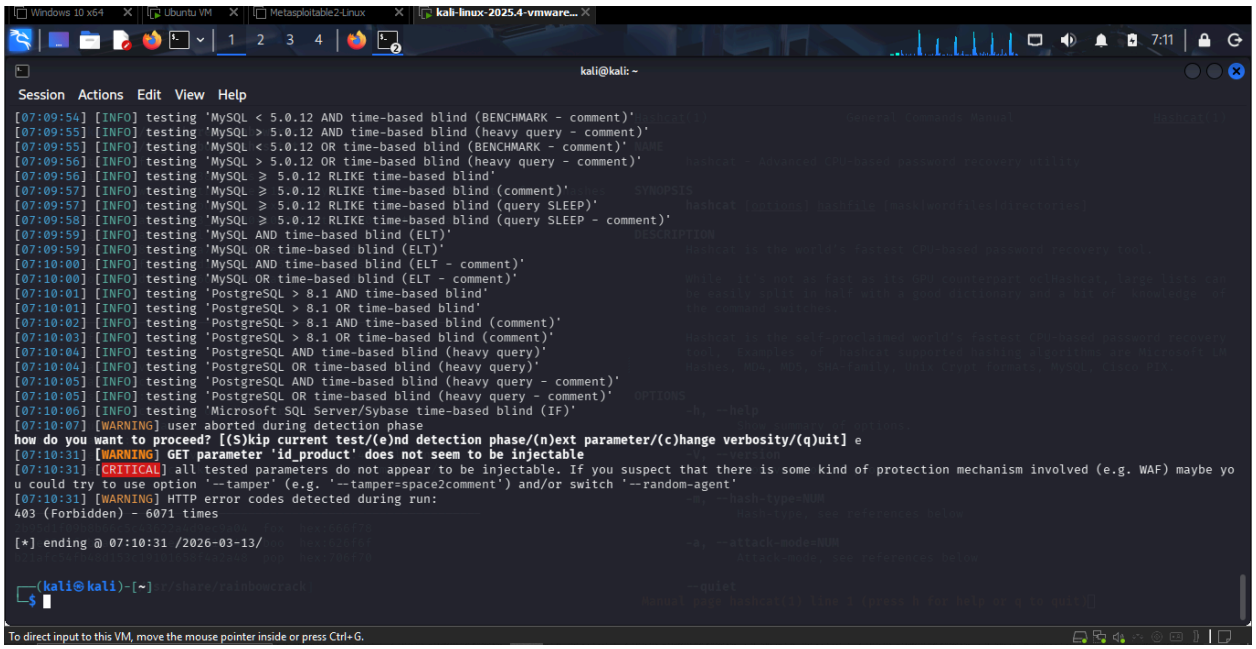
Step 3. Defense Implementation

To protect the web application, a Web Application Firewall was implemented using ModSecurity.The firewall was configured with custom security rules to detect common attack patterns such as SQL injection and directory traversal attempts.

The Example custom rule look like this,

SecRule ARGS "@detectSQLi" \
"id:110001,phase:2,deny,status:403,msg:'SQL Injection Attempt Blocked'"

When malicious payloads were sent to the application, the firewall intercepted and blocked the request, returning an HTTP 403 Forbidden response. Attached below is the Screenshot of WAF blocking attack



Note,when you take a very good look at the screenshot, you will see when the firewall intercepted and blocked the request by clearly returning an HTTP 403 forbidden response.

Step 4. Security Validation Results includes the test and results;

- The Directory Enumeration shows that Several directories discovered
- The SQL Injection Testing shows that no vulnerability detected
- The Automated Vulnerability Scan shows that Minor configuration issues were found
- The WAF Protection shows that it Successfully blocked malicious requests

The implemented security controls significantly reduced the attack surface of the application.

Step 5. Security Recommendations

The following security improvements are recommended:

- Enable HTTPS encryption to protect data in transit.
- Regularly update the PrestaShop application and install modules.
- Restrict access to administrative directories.
- Implement strong password policies for administrator accounts.
- Regularly review firewall logs to detect attack attempts.
- Disable unnecessary directories and services.

Conclusion

The security assessment demonstrated that the deployed PrestaShop application is reasonably secure against common web attacks. The implemented Web Application Firewall effectively blocked malicious traffic and prevented exploitation attempts.

However, additional security improvements such as HTTPS implementation and security header configuration would further strengthen the application’s security posture.